Experiment 6:

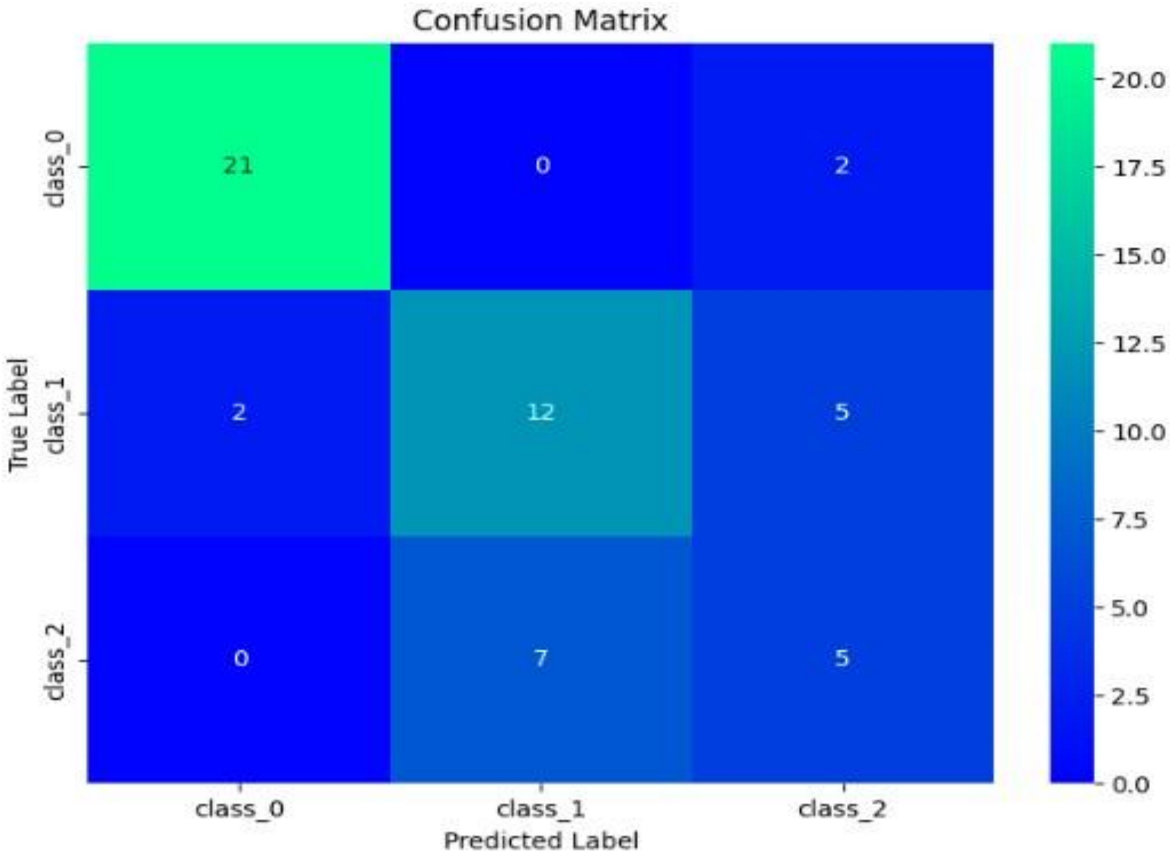
Program:

```
import pandas as pd
import pandas as pd
import
matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets
import load_wine from
sklearn.model_selection
import train_test_split
from sklearn.neighbors
import
KNeighborsClassifier
from sklearn.metrics
import
accuracy_score,confusi
on_matrix
wine=load_wine()
data=pd.DataFrame(dat
a=wine.data,columns=
wine.feature_names)
data['Target']=wine.targ
et
x=data.drop('Target',axi
s=1) y=data['Target']
```

```
x_train,x_test,y_train,y
_test=train_test_split(x,
y,test_size=0.3,random
_state=1)
model=KNeighborsClas
sifier(n_neighbors=5)
model.fit(x_train,y_trai
n)
y_pred=model.predict(x
_test)
accuracy=accuracy_sco
re(y_test,y_pred)
print("accuracy:",accura
cy) conf_matrix =
confusion_matrix(y_tes
t,y_pred)
plt.figure(figsize=(8,
6))
sns.heatmap(conf_matrix, annot=True, fmt='d',
cmap='summer', xticklabels=wine.target_names,
yticklabels=wine.target_names) plt.xlabel('Predicted
Label') plt.ylabel('True Label') plt.title('Confusion
Matrix') plt.show()
```

Output:

accuracy: 0.7037037037037037

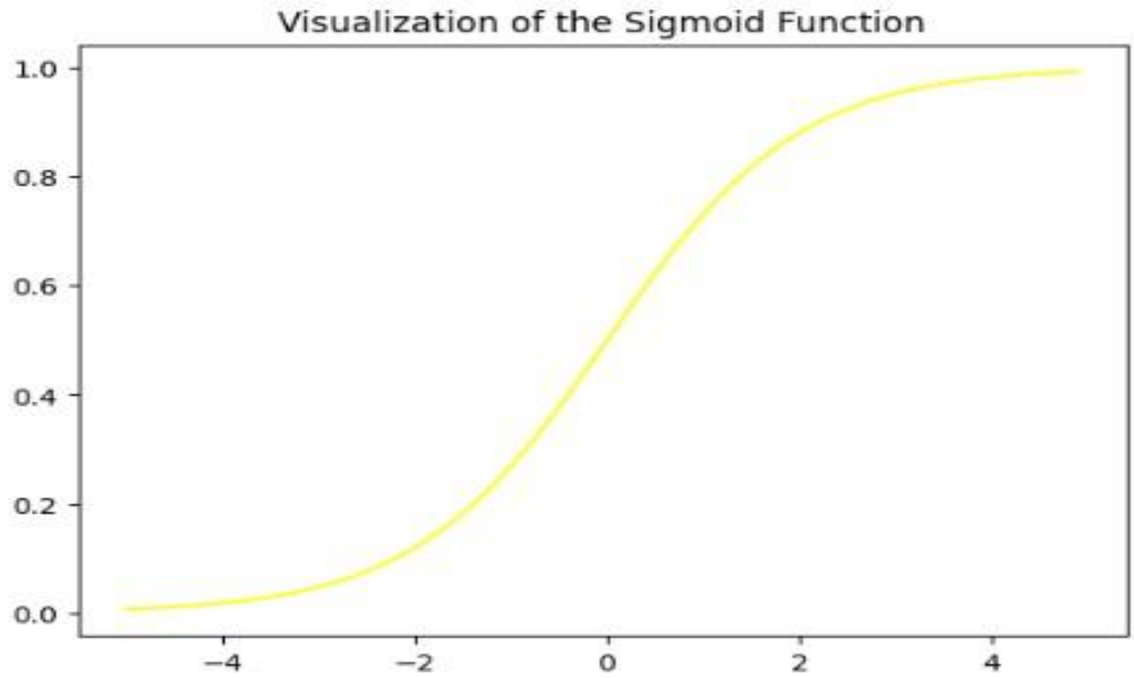


Experiment 7:

Program:

```
import numpy as np
import
matplotlib.pyplot as plt
def sigmoid(z):
    return 1 / (1 + np.exp( - z)) plt.plot(np.arange(-5, 5, 0.1),
sigmoid(np.arange(-5, 5, 0.1)),color='pink') plt.title('Visualization
of the Sigmoid Function') plt.show()
```

Output:



Experiment 8:

Program:

import pandas as pd import matplotlib.pyplot as plt

import seaborn as sns from sklearn.datasets import

load_iris from sklearn.model_selection import

train_test_split from sklearn.neighbors import

KNeighborsClassifier from sklearn.metrics import

accuracy_score,confusion_matrix iris=load_iris()

data=pd.DataFrame(data=iris.data,columns=iris.feature_

names) data['Species']=iris.target

x=data.drop('Species',axis=1) y=data['Species']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size

=0.3,random_state=1)

model=KNeighborsClassifier(n_neighbors=5)

model.fit(x_train,y_train) y_pred=model.predict(x_test) accuracy=accuracy_score(y_test,y_pred)

print("accuracy:",accuracy) conf_matrix =

confusion_matrix(y_test,y_pred) plt.figure(figsize=(8,

6))

sns.heatmap(conf_matrix, annot=True, fmt='d',

cmap='pink', xticklabels=iris.target_names,

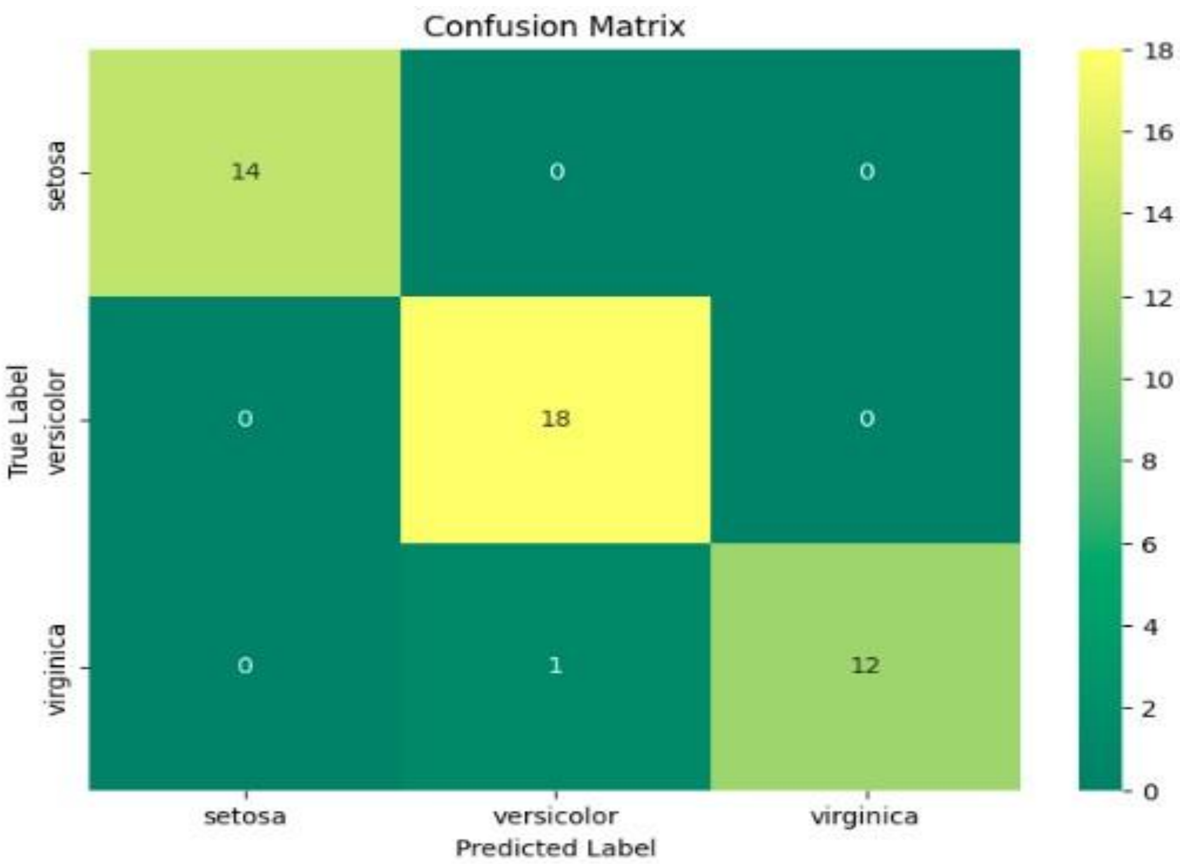
yticklabels=iris.target_names) plt.xlabel('Predicted

Label') plt.ylabel('True Label') plt.title('Confusion

Matrix') plt.show()

Output:

accuracy: 0.9777777777777777



Experiment 9:

Program:

#naive bayes iris import pandas as pd import

matplotlib.pyplot as plt import seaborn as sns from

sklearn.datasets import load_iris from

sklearn.model_selection import train_test_split from

sklearn.naive_bayes import GaussianNB from

sklearn.metrics import

accuracy_score,confusion_matrix iris = load_iris()

data = pd.DataFrame(data=iris.data,

columns=iris.feature_names) data['species'] = iris.target X

= data.drop('species', axis=1) y = data['species']

X_train, X_test, y_train, y_test = train_test_split(X, y,

test_size=0.3, random_state=42) model = GaussianNB()

model.fit(X_train, y_train) y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("accuracy:",accuracy) conf_matrix =

confusion_matrix(y_test,y_pred) plt.figure(figsize=(8, 6))

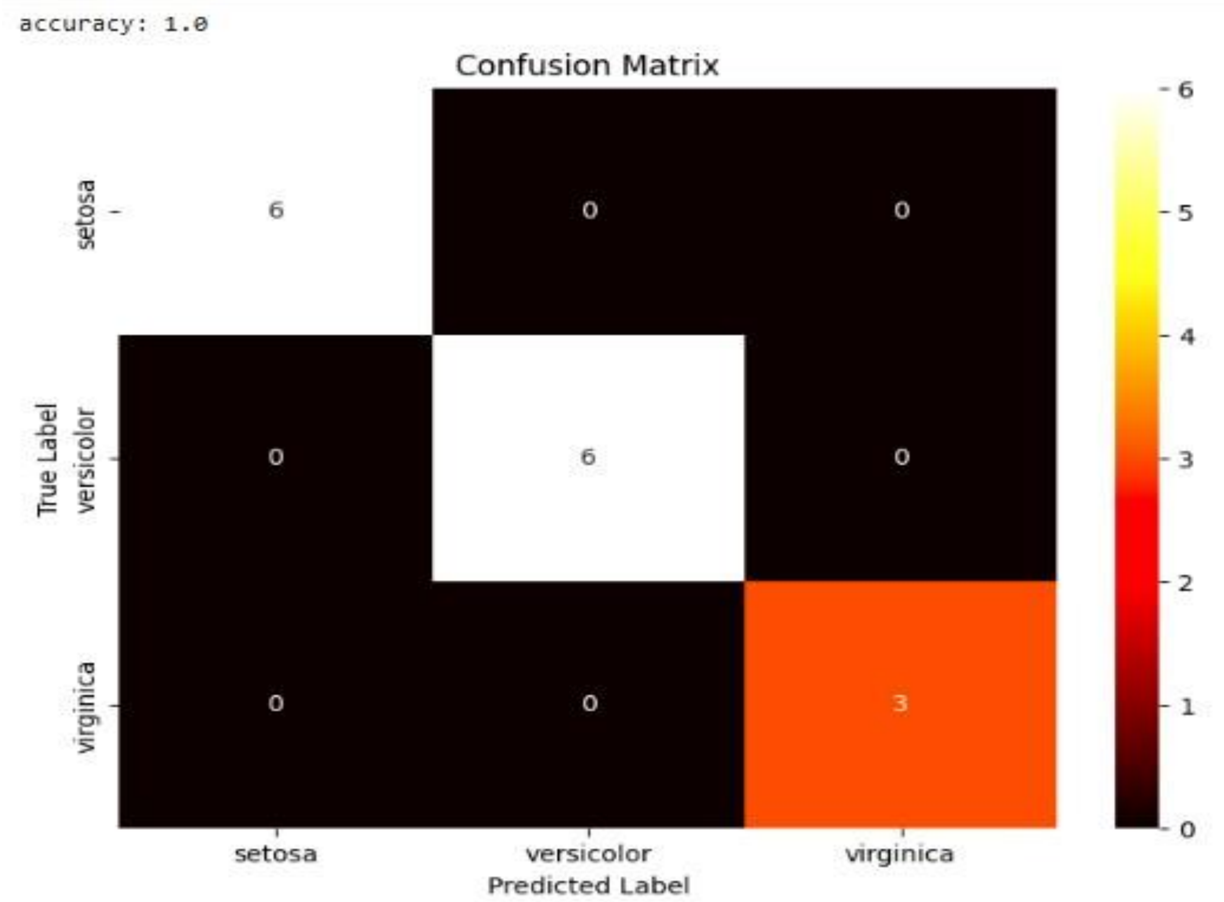
sns.heatmap(conf_matrix, annot=True, fmt='d',

cmap='vanimo', xticklabels=iris.target_names,

yticklabels=iris.target_names) plt.xlabel('Predicted

Label') plt.ylabel('True Label') plt.title('Confusion

Matrix') plt.show() **Output:** accuracy: 1.0



Experiment 10:

Program:

import pandas as pd import matplotlib.pyplot as plt import seaborn as sns from sklearn.datasets import load_iris from sklearn.model_selection import train_test_split from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score,confusion_matrix iris = load_iris() data = pd.DataFrame(data=iris.data, columns=iris.feature_names) data['species'] = iris.target X = data.drop('species', axis=1) y = data['species']

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=1) model
=LogisticRegression(max_iter=200) model.fit(X_train,
y_train) y_pred = model.predict(X_test) accuracy =
accuracy_score(y_test, y_pred) print("accuracy:",accuracy)
conf_matrix = confusion_matrix(y_test,y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d',
cmap='berlin', xticklabels=iris.target_names,
yticklabels=iris.target_names) plt.xlabel('Predicted
Label') plt.ylabel('True Label') plt.title('Confusion
Matrix') plt.show()
```

Output:

accuracy: 0.9777777777777777

```
accuracy: 0.9777777777777777
```